Rapport de stage (SPE)

2ème année Cycle Supérieur (2CS)

Option : Systèmes Intelligents et Données (SD)

Thème :

Engineering an Intelligent Centralized Web Portal for Data Center Virtual Machine Migration Tracking

Réalisé par :

- *HEMAIZIA Abderrahmene*

Encadré par :

- *BARKAT Siham*

Organisme d'accueil : Sonatrach TRC

Acknowledgments

I would like to express my deep gratitude to the academic supervisors at ESI for their continuous support and guidance throughout my studies. I am also very grateful to the professional supervisors at Sonatrach – Transpipeline Activities (TRC) for welcoming me to their team, providing a rich learning environment, and sharing their valuable expertise. Finally, I thank everyone who contributed directly or indirectly to the completion of this training report.

Abstract

Abstract:

This internship report, completed within the Solutions Métiers Department of the IT Direction (DTI) at SONATRACH TRC, addresses the critical challenges of data synchronization during Data Center migrations. Traditionally, recording Virtual Machine (VM) technical specifications during server relocations relied heavily on error-prone, manual Excel tracking. This project modernizes this workflow by designing and implementing an automated web portal featuring centralized form inputs and dynamic Excel exporting capabilities. Furthermore, aligning with the Intelligent Systems and Data (SD) curriculum, the platform incorporates intelligent modules for automated data ingestion, configuration error detection, and predictive resource allocation forecasting. This solution guarantees absolute data integrity, minimizes infrastructure migration downtime, and optimizes capacity planning for SONATRACH's enterprise server infrastructure.

Keywords: Automated Inventory, Data Center Migration, Express.js, Drizzle ORM, Dockerized PostgreSQL, SONATRACH TRC.

Table of Contents

Chapter 1: Project Context and Host Organization

1.1. Sonatrach Group Overview

Sonatrach (the National Company for Research, Production, Transport, Processing, and Marketing of Hydrocarbon Resources) is Algeria's national oil and gas company, owned by the state. Founded in 1963, it is the largest economic entity in Africa and a key player in the global energy sector. The company operates comprehensively across the hydrocarbon value chain, from exploration and production, through pipeline transport of oil and gas, to refining and petrochemical processing, and finally, logistics for international marketing.

1.2. Oil and Gas Pipeline Activity (TRC)

The Oil and Gas Pipeline Activity (TRC) is a vital operational branch within Sonatrach's corporate structure. Its primary mission is the safe, continuous, and optimal overland transport of strategic energy commodities across the national territory and to international export gateways [image_elXJRT.png]. The strategic commitments of the Transport Coordination Center branch include: Network infrastructure operations: managing thousands of kilometers of high-pressure pipelines transporting crude oil, condensate, LPG, and natural gas [image_elXJRT.png]. Offshore terminal logistics: organizing coastal storage facilities and marine shipping infrastructure dedicated to international export vessels [image_elXJRT.png]. Coordination of transcontinental flows: monitoring key international transport corridors, including the Enrico Mattei (Transmed) and Pedro Duran Farrell (Morocco-Europe) gas pipelines [image_IPFUUO].

1.3. Host section overview

This project was implemented within the Directorate of Information Technology (DTI), which provides centralized digital support for pipeline transport activities [image_6VFVE_]. The DTI is responsible for establishing corporate networks, maintaining industrial communications, and building unified computing environments [image_wiS7Nb]. The practical training took place within the Business Solutions Department [image_wiS7Nb]. This specialized department focuses on: Enterprise Software Engineering: developing and deploying enterprise software systems specifically designed to meet the operational requirements of pipeline logistics [image_MbO3j0]. Process Digitization: eliminating legacy manual workflows, introducing structured databases, and improving administrative flows between departments [image_MbO3j0]. Data Integration: standardizing information flows across remote regional sectors to ensure data accessibility and transparent business reporting [image_MbO3j0].

1.4. Project Context and Problem Statement

1.4.1. Data Center Migrations

To maintain high availability, scalability, and robust disaster recovery standards, the Data Migration Management (DTI) performs periodic modifications to the hardware infrastructure. These complex technical procedures include structured data center migrations, physical server upgrades, and the migration of hundreds of virtual machines across different physical computing nodes and cluster environments.

1.4.2. Current operational limitations

Historically, recording and inventorying infrastructure attributes during large-scale server migrations has relied on manual tracking. Technicians populate standalone spreadsheet files on desktop computers (such as Microsoft Excel spreadsheets) to index server hostnames, virtualization configurations, static IP addresses, memory allocations, and storage paths. This decentralized approach leads to several vulnerabilities: High probability of human error: Manual data entry results in syntax inconsistencies, spelling errors in IP tables, or inaccurate device names, which can delay server setup times. Version control fragmentation: Multi-user editing across spreadsheets shared via email is difficult to synchronize. This friction leads to data conflicts and incomplete device inventories. Lack of constraints: Standalone files do not provide automatic input validation, which can result in duplicate network address entries or incorrect environment markers.

1.4.3. Updated solution

To address these operational risks, this project establishes an online management portal. The platform provides systems engineers with a single, multi-user input interface that utilizes real-time constraint validation. By linking all user inputs to a central database, data center administrators can view real-time migration status dashboards and export clean, audited Excel asset tables on demand.

Chapter 2: Analysis of the Existing System and Requirements Elaboration

2.0. Methodological Alignment with ESI Competency Framework

In strict accordance with the **ESI 2CS SPE Syllabus**, this engineering project is managed under two core competency families:

- **Client Need Analysis (CF1 / C1.2):** Addressed by capturing the operational realities of the SONATRACH TRC divisions and translating the chaotic spreadsheet workflow into structured system specifications.
- **Solution Engineering and Design (CF9 / C9.3):** Satisfied by executing an active third-normal-form (3NF) relational mapping, designing a full-stack automated JavaScript environment, and delivering verifiable deployment testing strategies.

2.1. Detailed Mapping of the Legacy Data Structure

To accurately transition from legacy document-based record-keeping to a relational database architecture, a comprehensive analysis of the official SONATRACH TRC "Technical Publication and Resource Provisioning Form" (*Formulaire Technique de Publication*) was conducted.

A critical operational characteristic of this workflow is that **each individual file represents a single, isolated Virtual Machine (VM)**. Consequently, a complete Data Center migration involves managing, filling, and tracking a massive volume of separate documents simultaneously. The structural fields within each form are organized into four core data dimensions:

2.1.1. Administrative Metadata & Custody Fields

- **Organizational Hub (*Pôle*)**: Identifies the regional center managing the infrastructure (e.g., ALGER).
- **Structure Name**: Specifies the internal business branch requesting the resource (e.g., TRC Siège / EXP).
- **Structure & Service Responsibles**: Pinpoints the managers in charge of the service deployment lifecycle.
- **Technical Contact**: Records the deployment engineer details for direct follow-ups (e.g., Berkat Siham).
- **Target Audience**: Outlines the operational users exploiting the published service (e.g., Agents of the EXP Directorate and regional exploitation divisions across 9 regional directions).

2.1.2. Core Network & Software Configuration Attributes

- **Network Identifiers**: Captures DNS Entries, Internal IP Addresses, Virtual F5 IPs, and Port bindings (e.g., Port 80, 443) assigned to that specific VM.
- **Operating System Environment**: Documents the underlying OS layers (e.g., Windows Server 2022, WSL Ubuntu Server 24.04).
- **Software Stack Matrix**: Maps checkboxes and versions for modern development frameworks and databases utilized within TRC DTI, including:
    - *Languages & Frameworks*: Python, Django, Node.js.
    - *Database Management Systems*: Oracle, MySQL, MongoDB, PostgreSQL.
    - *Web Servers & Proxies*: Nginx, Apache Tomcat.

2.1.3. Infrastructure Traffic & Flow Matrices (*Matrice des Flux*)

Each VM form catalogs its own dedicated multi-point network routing rules defining:

- **Source & Destination Zones**: Subnet blocks (e.g., 10.x.x.x) or user groups interacting with the VM.
- **Service Protocols & Ports**: Specific network protocols such as RDP/TCP (3389) and SSH/TCP (22) for infrastructure administration, alongside HTTP/TCP (80) and HTTPS/TCP (443) for application web rendering.

2.1.4. Security Compliance & Verification Checkpoints

Managed strictly by the Information System Security Central Directorate (DSI/DG), these boolean checkpoints evaluate the VM's operational readiness before publication:

- DMZ Zone validation and up-to-date Antivirus signature verification.
- TLS/SSL certificate enforcement.
- Authenticated vulnerability scans, Web vulnerability evaluations, and Source Code audits.
- Compliance with the Principle of Least Privilege (PoLP).

2.2. Critical Diagnosis and Bottlenecks

The current method of executing Data Center migrations via hundreds of independent Excel documents triggers severe bottlenecks for the DTI Business Solutions Department:

- **Data Fragmentation and Silos**: Because each VM configuration is locked inside its own individual spreadsheet file, it is impossible for network administrators to get a holistic view of the global infrastructure status or calculate total resource consumption easily.

- **Cross-Form Inconsistencies**: Manual sheets lack global synchronization. An engineer could easily allocate a duplicate IP address or bind an already restricted port on two different VM files without knowing it, leading to critical network conflicts during deployment.
- **Version Control Chaos**: During large-scale migrations involving multiple teams, updating hundreds of separate files leads to data loss, outdated machine inventories, and uncoordinated security audits.

2.3. System Requirements Elaboration

2.3.1. Functional Requirements (FR)

- **FR-1: Multi-Tenant Role-Based Access Control (RBAC)**: Enforce separate workspaces for the Requesting Structure (data entry), the DTI Dev team (platform management), and the Security Directorate (compliance check updates).
- **FR-2: Centralized Bulk VM Management Portal**: A unified interface allowing users to manage multiple VM records simultaneously under a single "Migration Project" instead of opening separate files.
- **FR-3: Dynamic Publication Form**: An intuitive digital interface to input individual VM attributes, featuring nested toggle grids for software tech stack selection and dynamic entry rows for network flow matrices.
- **FR-4: Automated Inter-VM Constraint Validation**: Automatic collision detection preventing duplicate IP allocations, hostname conflicts, or unauthorized port configurations across all active VM registration forms.
- **FR-5: Verified Asset Exporting**: One-click generation tool converting selected digital VM records into approved corporate Excel spreadsheets matching SONATRACH's exact layouts.
- **FR-6: Infrastructure Environment Segregation:** The portal must provide explicit classification tools to tag each virtual machine node under distinct lifecycle environments (e.g., Development, Testing, Production).

2.3.2. Non-Functional Requirements (NFR)

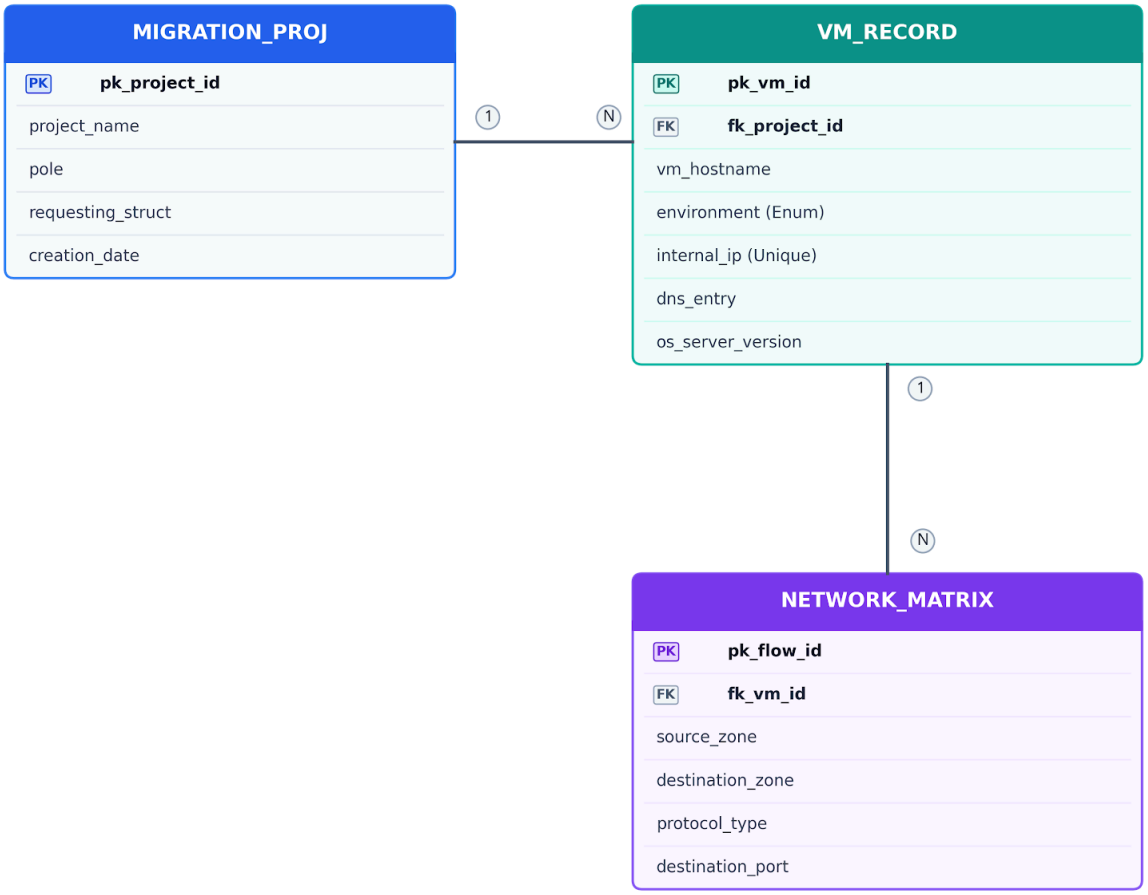
- **NFR-1: Cybersecurity Compliance**: Data transitions and session stores must rely on cryptographic standards matching TRC security requirements (such as password hashing and input sanitization to block SQL injections).
- **NFR-2: Scalability and Performance**: The system database must efficiently handle hundreds of concurrent VM records and flow matrices, ensuring dashboard UIs load under 2 seconds during peak migration sessions across regional operational offices.

- **NFR-3: Data Normalization**: The underlying database must decouple the "Migration Project", the "VM Specifications", and the "Network Flows" into distinct tables to eliminate redundancy.

Chapter 3: Conception and System Architecture

3.1. Relational Database Modeling & Normalization

To resolve the data fragmentation caused by hundreds of isolated Excel spreadsheets, a unified relational database schema is engineered. The architecture decouples projects, individual virtual machines, software stacks, and network matrices into a normalized schema (3NF) to ensure data integrity and prevent update anomalies.



1. MIGRATION_PROJ Table (Data Center Migration Projects)

- pk_project_id (Data Type: INT / Serial): Auto-incrementing primary key. Unique identifier for each physical or virtual infrastructure relocation project.
- project_name (Data Type: VARCHAR(100)): NOT NULL constraint. Descriptively titles the global infrastructure shift operation.
- pole (Data Type: VARCHAR(255)): NOT NULL constraint. Specifies the handling regional SONATRACH TRC hub (e.g., ALGER, ARZEW, SKIKDA).
- requesting_struct (Data Type: VARCHAR(100)): NOT NULL constraint. Tracks the business branch requesting the resources (e.g., TRC Siège / EXP).

- creation_date (Data Type: TIMESTAMP): NOT NULL constraint. System-generated timestamp logging when the migration record was initialized.

2. VM_RECORD Table (Virtual Machine Inventory Nodes)

- pk_vm_id (Data Type: INT / Serial): Primary key identifying a single virtual machine entity.
- fk_project_id (Data Type: INT): Foreign key (FK) referencing MIGRATION_PROJ(pk_project_id). Configured with a CASCADE DELETE constraint to ensure structural cleanup if a project is dropped.
- vm_hostname (Data Type: VARCHAR(100)): NOT NULL constraint. Specifies the unique computing network hostname within the corporate domain.
- environment (Data Type: ENUM('DEV', 'TEST', 'PROD')): NOT NULL constraint. Discriminant variable that partitions virtual machines by operational lifecycle criticality. This variable enables asymmetric data routing validations inside the SD validation engine.
- internal_ip (Data Type: VARCHAR(15)): Enforced UNIQUE and NOT NULL constraints. Represents the physical internal static IPv4 address. Network collisions are natively intercepted by the database constraints.
- dns_entry (Data Type: VARCHAR(100)): Dedicated uniform resource locator mapping to the published web workflow (e.g., intranet.trc.sonatrach.dz).
- virtual_f5_ip (Data Type: VARCHAR(15)): Tracks the front-facing high-availability load balancer entry points assigned to high-tier PROD systems.
- os_server_version (Data Type: VARCHAR(50)): Logs the kernel or base operating system distribution version (e.g., Windows Server 2022, Ubuntu 24.04).

3. NETWORK_MATRIX Table (Infrastructure Traffic & Firewalls)

- pk_flow_id (Data Type: INT / Serial): Auto-incrementing primary key uniquely tagging a precise firewall network routing rule.
- fk_vm_id (Data Type: INT): Foreign key (FK) pointing directly to VM_RECORD(pk_vm_id). Maps a granular matrix rule to its layout server node.
- **`source_zone`** (Data Type: `VARCHAR(50)`): `NOT NULL` constraint. Defines the subnetwork domain generating the traffic sequence (e.g., `10.x.x.x`, `UTILISATEURS`).
- **`destination_zone`** (Data Type: `VARCHAR(50)`): `NOT NULL` constraint. Points to the receiving endpoint resource (generally resolves to the VM target).
- **`protocol_type`** (Data Type: `VARCHAR(10)`): `NOT NULL` constraint. Represents the network transport protocol layer used (e.g., `TCP`, `UDP`).
- **`destination_port`** (Data Type: `INT`): `NOT NULL` constraint. Pinpoints the listening daemon network gateway port (e.g., `22` for SSH, `80` for HTTP, `443` for HTTPS, `3389` for RDP).

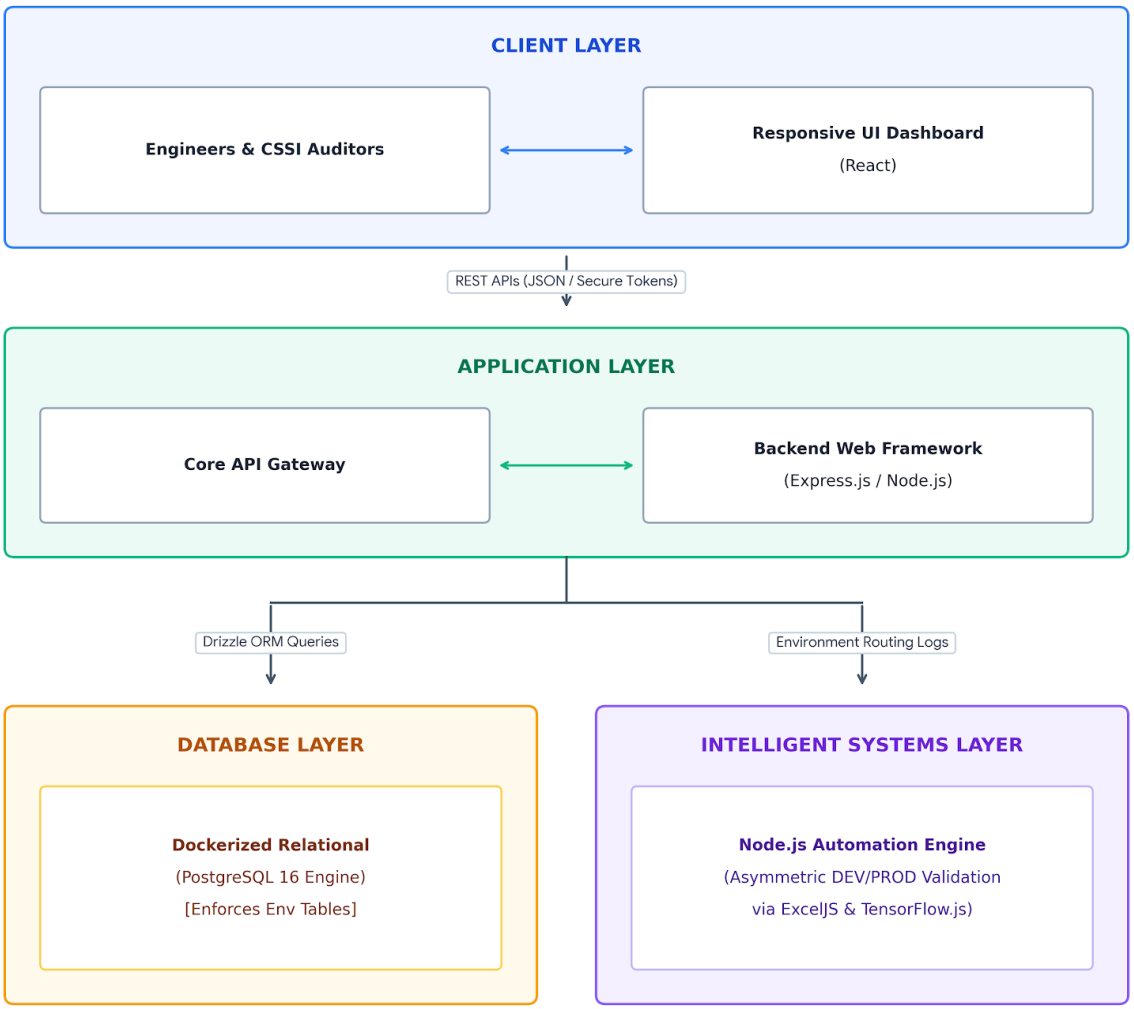
- **`flow_description`** (Data Type: `TEXT`): Human-readable comment detailing the flow's utility (e.g., `Administrative RDP Access`).

3.1.1. Entity-Relationship Data Dictionary

- **MIGRATION_PROJECT Table**: Holds administrative context metadata. Centralizes tracking for a data center relocation hub. Fields include pk_project_id, project_name, pole, requesting_structure, and creation_date.
- **VM_RECORD Table**: Represents individual server nodes. Replaces the core fields of the manual form. Fields include pk_vm_id, fk_project_id (Foreign Key), vm_hostname, environment (**Enforced Enum: DEV, TEST, PROD**), internal_ip (Enforced Unique Constraint), dns_entry, virtual_f5_ip, and os_server_version.
- **SOFTWARE_STACK Table**: A normalized lookup table referencing specific technical setups. Linked via a many-to-many relationship (VM_SOFTWARE_MAP) to index technical frameworks (e.g., Python, Django, Node.js, Oracle).
- **NETWORK_MATRIX Table**: Handles multi-point routing structures per virtual machine. Fields include pk_flow_id, fk_vm_id (Foreign Key), source_zone, destination_zone, protocol_type (TCP/UDP), destination_port, and flow_description.
- **SECURITY_COMPLIANCE Table**: Enforces a strict 1-to-1 relationship with VM_RECORD, allowing the Information System Security Directorate (CSSI) to toggle validation flags (e.g., is_dmz_validated, is_antivirus_active, tls_certified, vulnerability_scan_passed).

3.2. Software Engineering Architecture & Technology Stack

The platform implements a modern, decoupled architecture to ensure secure multi-user execution, reliable performance across regional divisions, and seamless automation capabilities.



3.2.1. Layer Implementations

- **Frontend Client Layer**: Built using **React.js** styled with corporate components. Delivers a responsive dashboard interface. Replaces complex Excel cells with adaptive input layouts, dynamic form row insertions for network flows, and multi-tenant authorization views.
- **Backend Application Layer**: Powered by **Express.js (Node.js)** running in a strict **TypeScript** environment. The application utilizes **Drizzle ORM** for type-safe database queries, automated SQL schema migrations, and high-performance transactional execution. Express handles multi-tenant authorization middleware, runtime input validation, dynamic Excel generation logic, and exposes secure REST APIs for frontend synchronization.

- **Persistent Storage Layer**: Utilizing a **PostgreSQL** relational engine. Enforces data typing rules, asset dependency constraints, and concurrent transaction locking during multi-user migration operations.

3.3. Intelligent Systems & Data (SD) Integration Modules

To elevate this platform from a generic data-entry application to an intelligent asset platform, three automated data modules are integrated within the system backend:

3.3.1. Intelligent Legacy Bulk Ingestor (Parsing Engine)

Instead of forcing engineers to manually transcribe old data center spreadsheets into the portal, a Node.js automation pipeline utilizes **ExcelJS** (or **SheetJS**) data parsing libraries and data manipulation libraries.

- **Mechanism**: The ingestion engine accepts batches of legacy Excel files. It scans the unformatted matrices, uses regular expressions (Regex) to dynamically identify server name cells, software checkboxes, and IP flow strings, and transforms unstructured file layouts into valid SQL insert tuples.

3.3.2. Predictive Capacity and Allocation Planner

An intelligent analytics service runs predictive validations using **TensorFlow.js** linear regression models running natively within the Node.js runtime environment prior to server group migrations.

- **Mechanism**: A regression model evaluates aggregate compute payloads. The planner segregates metrics based on the environment tag: it allows rule-based CPU/RAM overcommitting triggers for low-criticality **Development** clusters while maintaining absolute zero-saturation alerts for strategic **Production** targets during target host evaluations.

3.3.3. Algorithmic Data Validation and Collision Sentry

An automated validation daemon scans runtime configurations across all stored virtual machine entities.

- **Mechanism**: Employs structural matching rules and mathematical network masks (CIDR calculations). It dynamically blocks invalid cross-environment network routes, raising high-severity alarms if a virtual machine tagged as **Development** requests a direct publication flow into a restricted **Production** database zone.

Chapter 4: System Implementation and Interface Design

4.1. Software Deployment Environment and System Configurations

The engineered web management platform is deployed in a secure staging environment within the SONATRACH TRC DTI private enterprise network. The application runtime leverages Docker containers to isolate infrastructure services, ensuring fast startup times and consistent environments between development and production.

4.1.1. Core Runtime Configurations

- Web Server Layer: An Nginx reverse proxy acts as the front-facing gateway, handling SSL/TLS termination and efficiently serving static React compiled bundles.
- Application Gateway: **PM2 (Process Manager 2)** serves as the production cluster manager for Node.js, ensuring application uptime, automatic restarts, and routing incoming client requests directly to the active Express.js instances.
- Database Engine: A containerized PostgreSQL 16 instance manages transactional persistence, executing automated schema migrations and indexing unique IP constraints.

4.2. Core Functional Interface Layouts

To replace the legacy process of managing hundreds of individual Excel tracking forms, the platform provides an intuitive, web-based dashboard split into three core user interfaces.

4.2.1. The Unified Migration Project Dashboard

This interface serves as the entry point for infrastructure managers. It displays a comprehensive summary of all ongoing Data Center migrations across different regional hubs (*Pôles*).

- Visual Elements: A clean data grid showing the project name, requesting department (e.g., TRC Siège / EXP), total virtual machines involved, and a real-time progress bar.
- Interactive Actions: Users can create a new migration project, assign technical contacts, or click on an active project to view its individual server inventory.

4.2.2. The Centralized Multi-VM Entry Portal

This dashboard replaces the manual Excel file fields with an integrated digital workflow. It allows system engineers to record configurations for multiple Virtual Machines inside a single workspace.

- Section A: Section A: System Identification: Form inputs capture metadata including hostname, primary DNS string, an environment selection toggle switch (DEV / PROD), deployment zone (e.g., DMZ vs. Internal Corporate LAN), and underlying operating system layers
- Section B: Dynamic Software Stack Selector: An adaptive checkbox grid displays categorical runtime packages. Selecting a primary component (e.g., Python) automatically exposes a nested sub-form to select framework extensions (e.g., Django) and version parameters.
- Section C: The Dynamic Traffic Flow Matrix: Engineers map out network rules using a flexible, spreadsheet-style table layout. A single button allows technicians to append infinite routing entries (defining Source, Destination, Protocol, and Port boundaries) without reloading the page.

4.2.3. The CSSI Compliance Audit Console

Designed specifically for the Information System Security Central Directorate, this access-restricted workspace indexes submitted virtual machine records awaiting production release.

- Security Checklists: Evaluators check active cryptographic indicators, cross-reference vulnerability scanning status codes, and audit privilege roles.
- One-Click Corporate Export: Once compliance requirements are met, the auditor clicks an "Export Verified Asset Sheet" button.The backend instantly triggers an **ExcelJS asynchronous utility block** that compiles the normalized database rows... that compiles the normalized database rows into an approved, multi-tab Excel spreadsheet matching SONATRACH's layouts.

Conclusion and Future Perspectives

Final Synthesis

This project successfully modernizes the enterprise infrastructure lifecycle within the **SONATRACH TRC DTI Business Solutions Department**. By migrating from hundreds of disconnected spreadsheet files to a secure, relational database web application, the platform eliminates severe human entry errors, halts cross-form data duplication, and establishes clear collaboration boundaries between system deployment teams and security auditors.

Furthermore, integrating data engineering tools—such as intelligent legacy parsing scripts and algorithmic validation rules—provides a practical showcase of the skills learned in the **Intelligent Systems and Data (SD)** curriculum at ESI.

Future Enhancements

- **Active Directory (LDAP) Integration**: Linking user sessions directly with SONATRACH's central directory services to enforce absolute corporate audit trails.
- **Direct Hypervisor Orchestration**: Developing backend connectors to communicate directly with active virtualization clusters (e.g., VMware vSphere or Nutanix API layers) to automatically fetch actual configuration attributes in real time, removing manual data entry entirely

References (Norme APA 7th Edition)

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].

Drizzle Team. (2026). Drizzle ORM documentation: TypeScript SQL ORM [Online documentation]. https://drizzle.team

Express Team. (2026). Express: Fast, unopinionated, minimalist web framework for Node.js [Online documentation]. https://expressjs.com

ExcelJS Project. (2025). ExcelJS: Excel Workbook Manager for Node.js and Browser [Open-source repository]. GitHub. https://github.com

SONATRACH TRC. (2023). Formulaire technique de publication et de mise à disposition des ressources (Rev 1.0) [Internal Corporate Documentation]. Direction Générale / Direction Centrale Informatique et Système d'Information.